

Análise Sintática Descendente

Parser recursivo, FIRST/FOLLOW e gramáticas LL(1)

Objetivo da aula

Compreender como um parser descendente constrói a análise a partir do símbolo inicial, como eliminar obstáculos à predição e como usar FIRST, FOLLOW e tabelas LL(1) para reconhecer programas da linguagem MINIC.

Conexão com a Aula 6

A gramática livre de contexto forneceu as regras da sintaxe. Agora essas regras serão transformadas em um procedimento efetivo de reconhecimento e construção da árvore sintática.

Referência principal: Aho, Lam, Sethi e Ullman, *Compiladores: Princípios, Técnicas e Ferramentas*, 2ª ed., Capítulo 4, Seções 4.3 e 4.4, pp. 133–145.

1. O que é análise sintática descendente?

A análise sintática descendente constrói a árvore de derivação de cima para baixo: começa no símbolo inicial e expande os não-terminais até alcançar os tokens da entrada. O parser tenta reproduzir uma derivação mais à esquerda da sentença.

Em cada etapa, o desafio é escolher qual produção aplicar para o não-terminal que está sendo expandido. Um parser simples pode tentar alternativas e voltar atrás; um parser preditivo escolhe a produção usando o próximo token, sem retrocesso.

Fluxo de entrada: o analisador léxico entrega tokens; o parser consome esses tokens, verifica a estrutura e produz uma árvore ou AST.

Exemplo para expressões:

$$E \rightarrow T E' \quad E' \rightarrow + T E' \mid \varepsilon \quad T \rightarrow F T' \quad T' \rightarrow * F T' \mid \varepsilon \quad F \rightarrow (E) \mid \text{id}$$

Essa forma evita a recursão à esquerda direta e permite que o parser decida com base no próximo token.

2. Descida recursiva

Na descida recursiva, cada não-terminal da gramática corresponde a um procedimento. O procedimento verifica os tokens terminais e chama outros procedimentos quando encontra não-terminais.

Pseudocódigo conceitual para um não-terminal A:

```
procedure A(): escolher uma produção  $A \rightarrow X_1 X_2 \dots X_k$  para cada  $X_i$ : se  $X_i$  é não-terminal: chamar  $X_i()$   
senão: casar  $X_i$  com o próximo token se o casamento falhar: sinalizar erro
```

No projeto MINIC, os procedimentos podem ser organizados como `parseProgram`, `parseDeclaration`, `parseStatement`, `parseExpression` e outros. Cada procedimento deve consumir exatamente os tokens correspondentes à construção reconhecida.

Quando há retrocesso, o parser salva a posição de entrada antes de tentar uma alternativa. Se a alternativa falhar, restaura a posição e tenta a próxima. Essa estratégia é simples, mas pode ser custosa e difícil de controlar.

Imagem de apoio do livro

As cadeias $E, -E, -(E), \dots, -(\text{id} + \text{id})$ são todas formas sentenciais dessa gramática. Escrevemos $E \xRightarrow{*} -(\text{id} + \text{id})$ para indicar que $-(\text{id} + \text{id})$ pode ser derivado de E .

Em cada passo de uma derivação, existem duas escolhas a serem feitas. Precisamos escolher qual não-terminal substituir e, tendo feito essa escolha, devemos selecionar uma produção com esse não-terminal como cabeça. Por exemplo, a derivação alternativa de $-(\text{id} + \text{id})$ a seguir difere da derivação (4.8) nos dois últimos passos:

$$E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E + E) \Rightarrow -(E + \text{id}) \Rightarrow -(\text{id} + \text{id}) \quad (4.9)$$

Cada não-terminal é substituído pelo mesmo corpo nas duas derivações, mas a ordem das substituições é diferente.

Para entender como os analisadores sintáticos funcionam, vamos considerar as derivações em que o não-terminal a ser substituído em cada passo é escolhido da seguinte maneira:

1. Em derivações *mais à esquerda*, o não-terminal mais à esquerda em cada forma sentencial sempre é escolhido. Se $\alpha \Rightarrow \beta$ é um passo em que o não-terminal mais à esquerda em α é substituído, escrevemos $\alpha \xRightarrow{lm} \beta$.
2. Em derivações *mais à direita*, o não-terminal mais à direita sempre é escolhido; escrevemos $\alpha \xRightarrow{rm} \beta$ nesse caso.

A derivação (4.8) corresponde à derivação mais à esquerda, e por isso pode ser reescrita como

$$E \xRightarrow{lm} -E \xRightarrow{lm} -(E) \xRightarrow{lm} -(E + E) \xRightarrow{lm} -(\text{id} + E) \xRightarrow{lm} -(\text{id} + \text{id})$$

Observe que (4.9) é uma derivação mais à direita.

Usando nossas convenções de notação, cada passo mais à esquerda pode ser escrito como $wA\gamma \xRightarrow{lm} w\delta\gamma$, onde w consiste apenas em terminais, $A \rightarrow \delta$ é a produção aplicada, e γ é uma cadeia de símbolos da gramática. Para enfatizar que α deriva β usando uma derivação mais à esquerda, escrevemos $\alpha \xRightarrow{lm} \beta$. Se $S \xRightarrow{lm} \alpha$, então dizemos que α é uma *forma sequencial mais à esquerda* da gramática dada.

Definições semelhantes são mantidas para derivações mais à direita. As derivações mais à direita às vezes são chamadas de derivações *canônicas*.

4.2.4 ÁRVORES DE DERIVAÇÃO E DERIVAÇÕES

Uma árvore de derivação é uma representação gráfica de uma derivação que filtra a ordem na qual as produções são aplicadas para substituir não-terminais. Cada nó interior de uma árvore de derivação representa a aplicação de uma produção. O nó interior é rotulado com o não-terminal A do lado esquerdo da produção; os filhos do nó são rotulados, da esquerda para a direita, pelos símbolos do corpo da produção pelo qual esse A foi substituído durante a derivação.

Por exemplo, a árvore de derivação para $-(\text{id} + \text{id})$, na Figura 4.3, resulta da derivação (4.8), bem como da derivação (4.9).

As folhas de uma árvore de derivação são rotuladas pelos não-terminais ou terminais e, lidas da esquerda para a direita, constituem uma forma sentencial, chamada *yield* ou *fronteira* da árvore.

Para ver o relacionamento entre derivações e árvores de derivação, considere qualquer derivação $\alpha_1 \Rightarrow \alpha_2 \Rightarrow \dots \Rightarrow \alpha_n$, onde α_1 é um único não-terminal A . Para cada forma sentencial α_i da derivação, é possível construir uma árvore de derivação cuja *yield* seja α_i . O processo é uma indução em i .

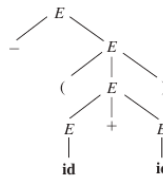


FIGURA 4.3 Árvore de derivação para $-(\text{id} + \text{id})$.

BASE: A árvore para $\alpha_1 = A$ é um único nó rotulado com A .

INDUÇÃO: Suponha que já tenhamos construído uma árvore de derivação com *yield* $\alpha_{i-1} = X_1 X_2 \dots X_k$ (observe que, de acordo com nossas convenções de notação, cada símbolo da gramática X_i é um não-terminal ou um terminal). Suponha que α_i seja derivado de α_{i-1} substituindo X_j um não-terminal, por $\beta = Y_1 Y_2 \dots Y_m$. Ou seja, no i -ésimo passo da derivação, a produção $X_j \rightarrow \beta$ é aplicada a α_{i-1} para derivar $\alpha_i = X_1 X_2 \dots X_{j-1} \beta X_{j+1} \dots X_k$.

3. Recursão à esquerda

Uma gramática é recursiva à esquerda quando um não-terminal pode derivar uma forma cujo primeiro símbolo é ele próprio. A forma direta mais comum é:

$$E \rightarrow E + T \mid T$$

Um parser descendente que chama `parseE()` e, imediatamente, tenta expandir `E` por `E + T` entra em chamadas infinitas sem consumir nenhum token.

A transformação padrão para eliminar a recursão à esquerda direta é:

$$A \rightarrow A \alpha \mid \beta \quad A \rightarrow \beta A' \quad A' \rightarrow \alpha A' \mid \epsilon$$

Aplicando à gramática de expressões:

$$E \rightarrow T E' \quad E' \rightarrow + T E' \mid \epsilon$$

A nova gramática preserva a linguagem, mas muda a forma da derivação para torná-la compatível com a análise descendente.

Imagem de apoio do livro



Recuperação no modo pânico

Com esse método, ao detectar um erro, o analisador sintático descarta um símbolo da entrada de cada vez até que um dentre um conjunto de *tokens de sincronismo* seja encontrado. Os tokens de sincronismo normalmente são delimitadores, como o ponto-e-vírgula ou `}`, cujo papel no programa fonte é claro e não ambíguo. O projetista do compilador deve selecionar os tokens de sincronismo de acordo com a linguagem fonte. Embora a correção no modo pânico normalmente ignore uma quantidade considerável de símbolos terminais do programa fonte sem se preocupar com a busca de erros adicionais, ele tem a vantagem da simplicidade e, diferentemente de alguns métodos que serão considerados mais tarde, tem-se a garantia de não entrar em um *loop* infinito.

Recuperação em nível de frase

Ao detectar um erro, um analisador sintático pode realizar a correção local sobre o restante da entrada; ou seja, pode substituir o prefixo da entrada restante por alguma cadeia que permita a continuação da análise. Uma correção local típica compreende a substituição de uma vírgula por um ponto-e-vírgula, a exclusão de um ponto-e-vírgula desnecessário, ou a inserção de um ponto-e-vírgula. A escolha da correção local fica a critério do projetista do compilador. Naturalmente, precisamos ter cuidado para que as substituições não provoquem *loops* infinitos, como aconteceria, por exemplo, se sempre inseríssemos algo na frente do símbolo da entrada corrente.

A substituição em nível de frase tem sido usada em diversos compiladores com recuperação de erro, pois pode corrigir qualquer cadeia da entrada. Sua principal desvantagem é a dificuldade de lidar com situações em que o erro real ocorreu antes do ponto de detecção.

Produções de erro

Nesta estratégia de recuperação de erro podemos entender a gramática da linguagem em mãos com produções que geram construções erradas, antecipando assim os erros mais comuns que poderiam ser encontrados em um programa. Um analisador sintático construído a partir de uma gramática estendida por essas produções de erro detecta os erros antecipadamente quando uma produção de erro é usada durante a análise. O analisador sintático pode, então, gerar um diagnóstico de erro apropriado sobre a construção errônea que foi reconhecida na entrada.

Correção global

Idealmente, gostaríamos que um compilador fizesse o mínimo de mudanças possível no processamento de uma cadeia de entrada incorreta. Existem algoritmos que auxiliam na escolha de uma sequência mínima de mudanças a fim de obter uma correção com um custo global menor. Dada uma cadeia incorreta x na entrada e uma gramática G , esses algoritmos encontram uma árvore de derivação para uma cadeia relacionada y , tal que o número de inserções, exclusões e substituições de tokens necessários para transformar x em y seja o menor possível. Infelizmente, esses métodos em geral são muito caros em termos de tempo e espaço de implementação, de modo que essas técnicas têm atualmente interesse meramente teórico.

Observe que o programa mais próximo do correto pode não ser aquele que o programador tinha em mente. Apesar disso, a noção de correção menos dispendiosa oferece uma medida para avaliar as técnicas de recuperação de erro, e tem sido usada para encontrar cadeias de substituição ideais para a recuperação em nível de frase.

4.2 GRAMÁTICAS LIVRES DE CONTEXTO

As gramáticas foram introduzidas na Seção 2.2 para descrever sistematicamente a sintaxe das construções de linguagem de programação como expressões e comandos. Usando uma variável sintática *stmt* para representar comandos e a variável *expr* para representar expressões, a produção

$$stmt \rightarrow \text{if}(expr) \text{ } stmt \text{ else } stmt \quad (4.4)$$

especifica a estrutura dessa forma de comando condicional. Outras produções definem exatamente o que é uma *expr* e o que mais um *stmt* pode ser.

Esta seção revê a definição de uma gramática livre de contexto e apresenta a terminologia adequada para falarmos sobre a análise sintática. Em particular, a noção de derivações é muito útil para discutirmos a ordem em que as produções são aplicadas durante o reconhecimento sintático.

4.2.1 A DEFINIÇÃO FORMAL DE UMA GRAMÁTICA LIVRE DE CONTEXTO

De acordo com a Seção 2.2, uma gramática livre de contexto (gramática, para abreviar) consiste em terminais, não-terminais, um símbolo inicial e produções.

1. *Terminais* são os símbolos básicos a partir dos quais as cadeias são formadas. O termo “nome de token” é um sinônimo para “terminal” e freqüentemente usaremos a palavra “token” em vez de terminal quando estiver claro que estamos

Discussão sobre transformação da gramática para favorecer analisadores preditivos, incluindo eliminação da recursão à esquerda e fatoração. Referência impressa: p. 137; PDF: p. 137.

4. Fatoração à esquerda

A fatoração à esquerda adia uma decisão quando duas alternativas começam pelo mesmo prefixo. Considere:

```
stmt → if expr then stmt else stmt | if expr then stmt
```

Ao ler apenas if, o parser ainda não sabe qual alternativa escolher. Fatoramos o prefixo comum:

```
stmt → if expr then stmt stmt' stmt' → else stmt | ε
```

A transformação permite consumir primeiro o prefixo compartilhado e decidir depois, quando o próximo token for conhecido.

Atenção ao “else vazio”: a fatoração pode remover o prefixo comum, mas não elimina necessariamente a ambiguidade. A linguagem e o parser precisam adotar uma regra, como associar o else ao then mais próximo.

Imagem de apoio do livro

- falando apenas sobre o nome do token. Assumimos que os terminais são os primeiros componentes dos tokens gerados pelo analisador léxico. Em (4.4), os terminais são as palavras-chave **if** e **else**, e os símbolos "(" e ")"
2. *Não-terminais* são variáveis sintáticas que representam conjuntos de cadeias. Em (4.4), *stmt* e *expr* são não-terminais. Os conjuntos de cadeias denotando não-terminais auxiliam na definição da linguagem gerada pela gramática. Os não-terminais impõem uma estrutura hierárquica sobre a linguagem que é a chave para a análise sintática e tradução.
 3. Em uma gramática, um não-terminal é distinguido como o *símbolo inicial*, e o conjunto de cadeias que ele representa é a linguagem gerada pela gramática. Por convenção, as produções para o símbolo inicial são listadas primeiro.
 4. As produções de uma gramática especificam a forma como os terminais e os não-terminais podem ser combinados para formar cadeias. Cada *produção* consiste em:
 - (a) Um não-terminal chamado de *cabeça* ou *lado esquerdo* da produção; essa produção define algumas das cadeias representadas pela cabeça.
 - (b) O símbolo $\rightarrow$. Às vezes, $:$ é usado no lugar da seta.
 - (c) Um *corpo* ou *lado direito da produção* consiste em zero ou mais terminais e não-terminais. Os componentes do corpo descrevem uma forma como as cadeias do não-terminal do lado esquerdo da produção podem ser construídas.

EXEMPLO 4.5: A gramática na Figura 4.2 define expressões aritméticas simples. Nessa gramática, os símbolos terminais são

id + - * / ()

Os símbolos não-terminais são *expression*, *term* e *factor*, e *expression* é o símbolo inicial da gramática.

<i>expression</i>	$\rightarrow$	<i>expression</i> + <i>term</i>
<i>expression</i>	$\rightarrow$	<i>expression</i> - <i>term</i>
<i>expression</i>	$\rightarrow$	<i>term</i>
<i>term</i>	$\rightarrow$	<i>term</i> * <i>factor</i>
<i>term</i>	$\rightarrow$	<i>term</i> / <i>factor</i>
<i>term</i>	$\rightarrow$	<i>factor</i>
<i>factor</i>	$\rightarrow$	(<i>expression</i>)
<i>factor</i>	$\rightarrow$	id

FIGURA 4.2 Gramática para expressões aritméticas simples.

4.2.2 CONVENÇÕES DE NOTACÃO

Para evitar ter de dizer sempre “estes são os terminais”, “estes são os não-terminais” etc., adotaremos as seguintes convenções de notação para as gramáticas no restante deste livro.

1. Estes símbolos são terminais:
 - (a) Letras minúsculas do início do alfabeto, como *a*, *b*, *c*.
 - (b) Símbolos operadores como +, * e assim por diante.
 - (c) Símbolos de pontuação como parênteses, vírgula e assim por diante.
 - (d) Os dígitos 0,1,...,9.
 - (e) Cadeias em negrito como **id** ou **if**, cada um representando um único símbolo terminal.
2. Estes símbolos são não-terminais:
 - (a) Letras maiúsculas do início do alfabeto, como *A*, *B*, *C*.
 - (b) A letra *S*, que, quando aparece, normalmente é o símbolo inicial da gramática.
 - (c) Nomes em minúsculas e itálico, como *expr* ou *stmt*.
 - (d) Nas construções das linguagens de programação, letras maiúsculas podem ser usadas para representar não-terminais. Por exemplo, os não-terminais para expressões, termos e fatores normalmente são representados por *E*, *T* e *F*, respectivamente.
3. Letras maiúsculas do fim do alfabeto, como *X*, *Y*, *Z*, representam símbolos da gramática, ou seja, não-terminais ou terminais.
4. Letras minúsculas do fim do alfabeto, principalmente *u*, *v*,..., *z*, representam cadeias de terminais, possivelmente vazias.
5. Letras gregas minúsculas, por exemplo α , β , γ , representam cadeias de símbolos da gramática, possivelmente vazias. Assim, uma produção genérica pode ser escrita como $A \rightarrow \alpha$, onde *A* é o lado esquerdo da produção ou a cabeça, e α representa o corpo ou lado direito da produção.

Exemplo de fatoração à esquerda e discussão de construções que não são resolvidas apenas pela transformação da gramática. Referência impressa: p. 138; PDF: p. 138.

5. FIRST e FOLLOW

As funções FIRST e FOLLOW ajudam a escolher a produção correta olhando apenas para o próximo token.

FIRST(α) é o conjunto de terminais que podem iniciar alguma cadeia derivada de α . Se α puder derivar a cadeia vazia, ϵ também pertence a FIRST(α).

FOLLOW(A) é o conjunto de terminais que podem aparecer imediatamente depois do não-terminal A em alguma forma sentencial. Para o símbolo inicial, o marcador de fim de entrada \$ pertence a FOLLOW.

Para a gramática de expressões:

$$E \rightarrow T E' \mid \epsilon \quad E' \rightarrow + T E' \mid \epsilon \quad T \rightarrow F T' \mid \epsilon \quad T' \rightarrow * F T' \mid \epsilon \quad F \rightarrow (E) \mid \text{id}$$

Alguns conjuntos importantes são:

Não-terminal	FIRST	FOLLOW
E	{ (, id }	{), \$ }
E'	{ +, ϵ }	{), \$ }
T	{ (, id }	{ +,), \$ }
T'	{ *, ϵ }	{ +,), \$ }
F	{ (, id }	{ *, +,), \$ }

Imagem de apoio do livro



Para modelar esse passo da derivação, encontre a j -ésima folha a partir da esquerda na árvore de derivação corrente. Essa folha é rotulada com X_j . Dê a essa folha m filhos, rotulados como $Y_1, Y_2, \dots, Y_m$, a partir da esquerda. Como um caso especial, se $m = 0$, então $\beta = \epsilon$, e atribuímos à j -ésima folha um filho rotulado com ϵ .

EXEMPLO 4.10: A sequência de árvores de derivação construída a partir da derivação (4.8) é mostrada na Figura 4.4. No primeiro passo da derivação, $E \Rightarrow -E$. Para modelar esse passo, acrescente dois filhos, rotulados com $-$ e E , à raiz E da árvore inicial. O resultado é a segunda árvore.

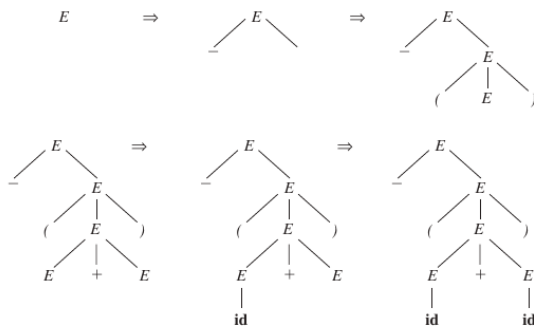


FIGURA 4.4 Sequência de árvores de derivação para a derivação (4.8).

No segundo passo da derivação, $-E \Rightarrow -(E)$. Consequentemente, inclua três filhos, rotulados com $(, E$ e $)$, à folha rotulada com E da segunda árvore, para obter a terceira árvore com fronteira $-(E)$. Continuando dessa forma, obtemos a árvore de derivação completa como a sexta árvore.

Como uma árvore de derivação ignora variações na ordem em que os símbolos nas formas sentenciais são substituídos, existe um relacionamento *multos-para-um* entre as derivações e as árvores de derivações. Por exemplo, as derivações (4.8) e (4.9) são associadas à mesma árvore de derivação final ilustrada na Figura 4.4.

No texto a seguir, freqüentemente faremos a análise sintática utilizando uma derivação mais à esquerda ou mais à direita, uma vez que há um relacionamento *um-para-um* entre as árvores de derivação e as derivações mais à esquerda ou mais à direita. Ambas as derivações escolhem uma ordem particular para substituir os símbolos nas formas sentenciais, de modo que também filtram variações na ordem. Não é difícil mostrar que toda árvore de derivação possui associada a ela uma única derivação mais à esquerda e uma única derivação mais à direita.

4.2.5 AMBIGÜIDADE

De acordo com a Seção 2.2.4, uma gramática que produz mais de uma árvore de derivação para alguma sentença é considerada *ambígua*. Colocando de outra forma, uma gramática livre de contexto é ambígua se permitir a construção de mais de uma derivação mais à esquerda ou mais de uma derivação mais à direita para a mesma sentença.

EXEMPLO 4.11: A gramática de expressão aritmética (4.3) permite duas derivações mais à esquerda distintas para a sentença $\text{id} + \text{id} * \text{id}$:

$$\begin{array}{ll}
 E \Rightarrow E + E & E \Rightarrow E * E \\
 \Rightarrow \text{id} + E & \Rightarrow E + E * E \\
 \Rightarrow \text{id} + E * E & \Rightarrow \text{id} + E * E \\
 \Rightarrow \text{id} + \text{id} * E & \Rightarrow \text{id} + \text{id} * E \\
 \Rightarrow \text{id} + \text{id} * \text{id} & \Rightarrow \text{id} + \text{id} * \text{id}
 \end{array}$$

A Figura 4.5 mostra as duas árvores de derivação correspondentes.

Observe que a árvore de derivação da Figura 4.5(a) reflete a precedência normalmente assumida dos operadores $+$ e $*$, o que não acontece na árvore da Figura 4.5(b). Ou seja, é comum tratar o operador $*$ como tendo maior precedência do que $+$, o que corresponde à avaliação de uma expressão do tipo $a + b * c$ como $a + (b * c)$, em vez de $(a + b) * c$.



6. Gramáticas LL(1)

Uma gramática LL(1) permite uma análise da entrada da esquerda para a direita, produzindo uma derivação mais à esquerda e usando um único token de lookahead para escolher a produção.

O parser é preditivo quando, para cada par “não-terminal + próximo token”, existe no máximo uma produção possível. Gramáticas com recursão à esquerda ou ambiguidade não são LL(1) em sua forma problemática.

Para uma produção $A \rightarrow \alpha$, a tabela preditiva é preenchida assim:

- Para cada terminal a em $FIRST(\alpha)$, inserir $A \rightarrow \alpha$ em $M[A, a]$.
- Se ϵ pertence a $FIRST(\alpha)$, inserir $A \rightarrow \epsilon$ para cada terminal em $FOLLOW(A)$.
- Uma entrada com duas produções indica conflito; a gramática não está adequada para essa tabela LL(1).

Exemplo de decisão:

Se o parser está expandindo F e o próximo token é id , escolhe $F \rightarrow id$. Se o próximo token é $($, escolhe $F \rightarrow (E)$. Qualquer outro token produz erro sintático.

Imagem de apoio do livro



regular também pode ser descrita por uma gramática, mas não vice-versa. Alternativamente, toda linguagem regular é uma linguagem livre de contexto, mas não vice-versa.

Por exemplo, a expressão regular $(ab)^n abb$ e a gramática livre de contexto

$$\begin{aligned} A_0 &\rightarrow aA_0 \mid bA_0 \mid aA_1 \\ A_1 &\rightarrow bA_2 \\ A_2 &\rightarrow bA_3 \\ A_3 &\rightarrow \epsilon \end{aligned}$$

descrevem a mesma linguagem, o conjunto de cadeias de as e bs terminando com abb .

Podemos construir mecanicamente uma gramática para reconhecer a mesma linguagem que um autômato finito não determinista (NFA). A gramática anterior foi definida a partir do NFA da Figura 3.24, usando a seguinte abordagem:

1. Para cada estado i do NFA, crie um não-terminal A_i .
2. Se o estado i possuir uma transição para o estado j lendo a , inclua a produção $A_i \rightarrow aA_j$. Se o estado i vai para o estado j lendo a entrada ϵ , inclua a produção $A_i \rightarrow A_j$.
3. Se i é um estado de aceitação, inclua $A_i \rightarrow \epsilon$.
4. Se i é o estado inicial, faça A_i ser o símbolo inicial da gramática.

Por outro lado, a linguagem $L = \{a^n b^n \mid n \geq 1\}$ com um número igual de as e bs é um exemplo típico de uma linguagem que pode ser descrita por uma gramática livre de contexto, mas não por uma expressão regular. Para ver por quê, suponha que L seja uma linguagem definida por alguma expressão regular. Suponha também que possamos construir um DFA D para aceitar L com um número finito de estados, digamos k . Como D tem apenas k estados, para uma entrada começando com mais do que k as , D deve entrar em algum estado duas vezes, digamos s_i , como na Figura 4.6. Suponha que o caminho de s_i de volta para si mesmo seja rotulado com uma sequência a^{j-i} . Como $a^j b^j$ está na linguagem, deve haver um caminho rotulado com b^j a partir de s_i para um estado de aceitação f . Mas há também um caminho a partir do estado inicial s_0 passando por s_i para f , rotulado com $a^j b^j$, como mostra a Figura 4.6. Portanto, D também aceita $a^j b^j$, que não está na linguagem, contradizendo a suposição de que L é a linguagem aceita por D .

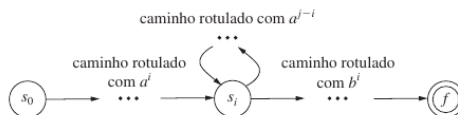


FIGURA 4.6 DFA D aceitando ambos $a^j b^j$ e $a^j b^j$.

Coloquialmente, dizemos que “autômatos finitos não sabem contar”, significando que um autômato finito não pode aceitar uma linguagem como $\{a^n b^n \mid n \geq 1\}$, que exigiria que ele mantivesse a contagem do número de as antes de ver os bs . Do mesmo modo, “uma gramática pode contar dois itens, mas não três”, conforme veremos quando considerarmos construções de linguagem dependente de contexto (ou sensível ao contexto), na Seção 4.3.5.

4.2.8 EXERCÍCIOS DA SEÇÃO 4.2

Exercício 4.2.1: Considere a gramática livre de contexto:

$$S \rightarrow SS \mid S S^* \mid a \text{ e a cadeia } aa + as.$$

- a) Dê uma derivação mais à esquerda para a cadeia.
- b) Dê uma derivação mais à direita para a cadeia.
- c) Dê uma árvore de derivação para a cadeia.
- ! d) A gramática é ambígua ou não ambígua? Justifique sua resposta.
- ! e) Descreva a linguagem gerada por essa gramática.

Exercício 4.2.2: Repita o Exercício 4.2.1 para cada uma das seguintes gramáticas e cadeias:

- a) $S \rightarrow 0S1 \mid 01$ com cadeia 000111.
- b) $S \rightarrow +SS \mid *SS \mid a$ com a cadeia $+ * aaa$.
- ! c) $S \rightarrow S(S)S \mid \epsilon$ com a cadeia $((()))$.
- ! d) $S \rightarrow S + S \mid SS \mid (S) \mid S * \mid a$ com a cadeia $(a + a) * a$.

7. Parser preditivo com tabela e pilha

Em vez de representar as chamadas de procedimentos diretamente na pilha de execução, um parser preditivo não recursivo mantém uma pilha explícita de símbolos da gramática.

Inicialmente, a pilha contém o símbolo inicial e o marcador \$. O parser consulta a tabela M usando o não-terminal no topo da pilha e o próximo token da entrada:

1. Se o topo é um terminal igual ao token corrente, desempilhar e avançar.
2. Se o topo é um não-terminal A, consultar $M[A, a]$ e substituir A pelo corpo da produção.
3. Se o topo é \$ e o token corrente é \$, aceitar.
4. Caso contrário, informar erro sintático.

A tabela transforma a gramática em um mecanismo executável. No MINIC, ela pode ser construída automaticamente a partir da gramática documentada, ou implementada como funções de descida recursiva quando a clareza didática for prioridade.

Imagem de apoio do livro

- ! e) $S \rightarrow (L) \mid a \text{ e } L \rightarrow L, S \mid S$ com a cadeia $((a,a),a,(a))$.
 !! f) $S \rightarrow aSbS \mid bSaS \mid \epsilon$ com a cadeia $aabbab$.
 ! g) A gramática a seguir para expressões booleanas:

$$\begin{aligned} bexpr &\rightarrow bexpr \text{ or } bterm \mid bterm \\ bterm &\rightarrow bterm \text{ and } bfactor \mid bfactor \\ bfactor &\rightarrow \text{not } bfactor \mid (bexpr) \mid \text{true} \mid \text{false} \end{aligned}$$

Exercício 4.2.3: Defina gramáticas para as seguintes linguagens:

- a) O conjunto de todas as cadeias de 0s e 1s tais que cada 0 seja imediatamente seguido por pelo menos um 1.
 ! b) O conjunto de todas as cadeias de 0s e 1s que sejam *palíndromos*, ou seja, a cadeia pode ser lida da mesma forma de trás para frente.
 ! c) O conjunto de todas as cadeias de 0s e 1s com um número igual de 0s e 1s.
 !! d) O conjunto de todas as cadeias de 0s e 1s com um número diferente de 0s e 1s.
 ! e) O conjunto de todas as cadeias de 0s e 1s em que 011 não apareça como uma subcadeia.
 !! f) O conjunto de todas as cadeias de 0s e 1s na forma xy , onde $x \neq y$ e x e y têm o mesmo tamanho.

!Exercício 4.2.4: Existe uma notação de gramática estendida muito usada na literatura. Nessa notação, colchetes e chaves ocorrendo do lado direito de uma produção são meta-símbolos (como $\rightarrow$ ou $\mid$) com os seguintes significados:

- i) Colchetes em torno de um símbolo ou símbolos da gramática indicam que essas construções são opcionais. Assim, a produção $A \rightarrow X[Y]Z$ tem o mesmo efeito das produções $A \rightarrow XYZ$ e $A \rightarrow XZ$.
 ii) Chaves ao redor de um símbolo ou símbolos da gramática dizem que esses símbolos podem ser repetidos qualquer número de vezes, incluindo zero vez. Assim, $A \rightarrow X\{YZ\}$ tem o mesmo efeito que a sequência infinita de produções-
 $A \rightarrow X, A \rightarrow XYZ, A \rightarrow XYZYZ, \text{ e assim por diante.}$

Mostre que essas duas extensões não aumentam o poder das gramáticas; ou seja, qualquer linguagem que possa ser gerada por uma gramática com essas extensões também pode ser gerada por uma gramática sem as extensões.

Exercício 4.2.5: Use as chaves e os colchetes descritos no Exercício 4.2.4 para simplificar a gramática a seguir para blocos de comandos e comandos condicionais:

$$\begin{aligned} stmt &\rightarrow \text{if } expr \text{ then } stmt \text{ else } stmt \\ &\mid \text{if } stmt \text{ then } stmt \\ &\mid \text{begin } stmtList \text{ end} \\ stmtList &\rightarrow stmt \mid stmtList \mid stmt \end{aligned}$$

! Exercício 4.2.6: Estenda a ideia do Exercício 4.2.4 para permitir qualquer expressão regular de símbolos da gramática no corpo de uma produção. Mostre que essa extensão não permite que as gramáticas definam nenhuma nova linguagem.

! Exercício 4.2.7: Um símbolo da gramática X (terminal ou não) é *inútil* se não houver derivações da forma $S \xRightarrow{*} wXy \xRightarrow{*} wxy$. Ou seja, X nunca aparecerá na derivação de nenhuma sentença.

- a) Dê um algoritmo para eliminar de uma gramática todas as produções contendo símbolos inúteis.
 b) Aplique seu algoritmo à gramática:

$$\begin{aligned} S &\rightarrow 0 \mid A \\ A &\rightarrow AB \\ B &\rightarrow 1 \end{aligned}$$

Exercício 4.2.8: A gramática da Figura 4.7 gera declarações para um único identificador numérico; essas declarações envolvem quatro propriedades diferentes e independentes de números.

$$\begin{aligned} stmt &\rightarrow \text{declare id optionList} \\ optionList &\rightarrow optionList \text{ option } \mid \epsilon \\ option &\rightarrow \text{mode} \mid \text{scale} \mid \text{precision} \mid \text{base} \\ mode &\rightarrow \text{real} \mid \text{complex} \\ scale &\rightarrow \text{fixed} \mid \text{floating} \\ precision &\rightarrow \text{single} \mid \text{double} \\ base &\rightarrow \text{binary} \mid \text{decimal} \end{aligned}$$

FIGURA 4.7 Uma gramática para declarações de múltiplos atributos.

8. Aplicação ao parser do MINIC

A equipe deve revisar a gramática da Etapa 2 e prepará-la para um parser descendente. O trabalho recomendado é:

1. Separar a gramática por níveis de precedência: expressão lógica, relacional, aditiva, multiplicativa e fator.
2. Eliminar recursões à esquerda.
3. Fatorar alternativas com prefixos comuns.
4. Definir ϵ apenas quando a ausência da construção for válida.
5. Calcular FIRST e FOLLOW dos não-terminais principais.
6. Implementar o parser e os testes de aceitação e rejeição.

Exemplo de decomposição de expressões:

```
expr → equality equality → relational ( ( == | != ) relational )* relational → additive ( ( < | <= | > | >= ) additive )* additive → term ( ( + | - ) term )* term → factor ( ( * | / ) factor )* factor → NUMBER | ID | ( expr )
```

Essa escrita com repetição é uma notação didática; na gramática usada pelo parser, cada repetição deve ser representada por produções equivalentes com não-terminais auxiliares.

Critério de sucesso: o parser deve consumir toda a entrada válida, construir a AST esperada e produzir mensagens úteis para entradas inválidas.

9. Atividades práticas

Atividade 1 — Eliminação de recursão: transforme $E \rightarrow E + T \mid T$ e $T \rightarrow T * F \mid F$ em uma gramática adequada para descida recursiva.

Atividade 2 — FIRST e FOLLOW: calcule FIRST e FOLLOW para a gramática transformada da Atividade 1. Inclua o marcador \$.

Atividade 3 — Tabela LL(1): preencha as entradas referentes a E, E', T, T' e F para os tokens id, (, +, *,), \$.

Atividade 4 — Implementação: implemente `parseExpression()`, `parseTerm()` e `parseFactor()`. Teste as entradas $x + y * 2$, $(x + y) * 2$ e $x + * 2$.

Atividade 5 — MINIC: escolha três construções da linguagem e registre: produção usada, tokens esperados, nó da AST produzido e mensagem de erro para uma entrada inválida.

Checklist da Etapa 2

- ☐ Gramática revisada e documentada.
- ☐ Recursão à esquerda eliminada.
- ☐ Alternativas fatoradas quando necessário.
- ☐ Parser consumindo tokens corretamente.
- ☐ AST construída para programas válidos.
- ☐ Erros sintáticos testados e localizados.

10. Síntese e leitura orientada

A análise sintática descendente parte do símbolo inicial e tenta construir uma derivação mais à esquerda. A descida recursiva associa procedimentos aos não-terminais; a eliminação da recursão à esquerda evita loops; a fatoração à esquerda adia decisões; FIRST e FOLLOW permitem construir escolhas preditivas; e a tabela LL(1) organiza essas escolhas de modo sistemático.

Na próxima aula, o parser será conectado à construção da AST e ao tratamento de erros sintáticos. A gramática deixa de ser apenas uma especificação e passa a orientar diretamente a implementação do projeto MINIC.

Leitura principal

Aho, A. V.; Lam, M. S.; Sethi, R.; Ullman, J. D. *Compiladores: Princípios, Técnicas e Ferramentas*. 2ª edição. Capítulo 4 — Seção 4.3, pp. 133–138, especialmente eliminação da recursão à esquerda e fatoração; Seção 4.4, pp. 138–145, especialmente descida recursiva, FIRST/FOLLOW, LL(1) e parser preditivo.

Referência das imagens

As imagens desta apostila são reproduções de páginas do livro fornecido para esta atividade e são utilizadas como apoio didático, com indicação da paginação impressa. Livro: *Compiladores: Princípios, Técnicas e Ferramentas*, Aho, Lam, Sethi e Ullman, 2ª ed.

Fim da Aula 7